

Contents

SPECIFICATION — Monte Carlo π Estimator (PyQt5 + matplotlib)	1
0. Intent and purpose	1
1. Actors and goals	2
2. Requirements (intent, high level)	2
3. Behavior and state model	3
3.1 System state machine (AppState)	3
3.2 Per-tick sampling behavior (the hot loop)	4
3.3 Threading model	4
3.4 The estimator and its uncertainty	4
4. Interfaces / contracts	4
C-01 MonteCarloEngine (pure, no GUI import — testable headless)	4
C-02 EstimationWorker(QThread)	5
C-03 Throttle contract	5
C-04 Data structures	5
5. UI specification	5
5.1 Top-level layout (MainWindow)	5
C-05 Scatter plot (matplotlib, embedded via FigureCanvasQTAgg)	6
C-06 Convergence plot	6
5.2 Control widgets and validation	6
6. Invariants (must hold in every valid implementation)	6
7. Constraints (precise and measurable — not prose)	7
8. Edge cases and failure semantics	7
9. Acceptance criteria, tests, and evals	8
9.1 Engine tests (deterministic, no Qt) — fast, run always	8
9.2 Statistical / probabilistic evals (the “estimator is correct” checks)	8
9.3 GUI / integration tests (offscreen, pytest-qt)	9
9.4 Manual / smoke eval (not automated; recorded)	9
10. Dependencies and environment	9
11. Traceability matrix (id -> where realized)	10

SPECIFICATION — Monte Carlo π Estimator (PyQt5 + matplotlib)

- **Status:** v0.1 — draft for implementation
- **Language:** Python 3.12 | GUI: PyQt5 | Plotting: matplotlib | Numerics: numpy
- **Scope of this document:** This is the *authoritative specification* of an AI-native system. It is written to Level 2–3 (structured, mostly executable): behavior, interfaces, invariants, edge cases, and failure semantics are made explicit so an agent (or engineer) can derive implementation **and** verification with minimal inference. Prose is used only where it adds intent; precision is applied everywhere it can be.
- **Principle:** requirements express *intent*; this specification *operationalizes* intent into observable behavior plus the conditions under which we know it is correct.

0. Intent and purpose

Estimate the mathematical constant π using the classical Monte Carlo “unit-square / quarter-circle” method, and present the estimate **live** in a graphical desktop application so the user can watch the stochastic estimate *converge* toward the true value.

Why this matters (context the agent should not re-derive): the value of a Monte Carlo estimate

is its *variance decay*, not a single number. Therefore the system’s primary product surface is not “print π ”; it is a **real-time convergence experience**: the running estimate, its uncertainty, its error versus true π , and the geometric picture of the sampling.

Non-goals (explicit, to constrain the solution space):

- No persistence, network, authentication, or multi-user features.
- No alternative π algorithms (Archimedes, Chudnovsky, BBP) — only Monte Carlo.
- No i18n / localization.
- No saving/loading of result files.

1. Actors and goals

Actor	Goals
User (human, single process)	Observe π being estimated from random samples at a chosen scale; control the simulation; reproduce a run via a seed.
Engine (MonteCarloEngine)	Given N and a batch budget, produce the next (deterministic) Monte Carlo estimate with no GUI dependencies.
Worker (EstimationWorker, a QThread)	Drive the engine off the UI thread, throttle progress, honor pause/resume/stop.
UI (MainWindow)	Present controls + two synchronized matplotlib charts; never block on computation.

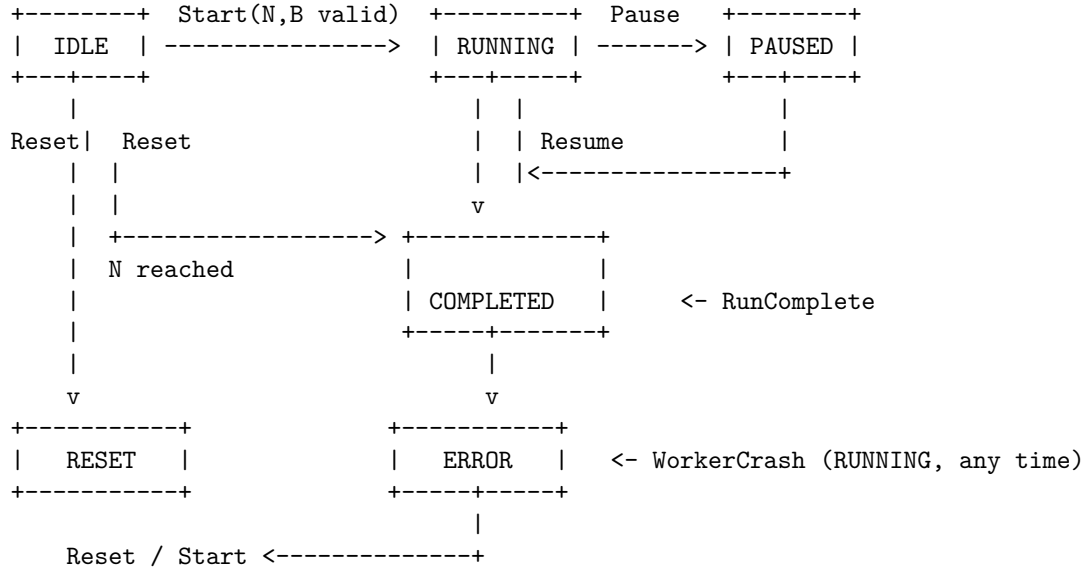
The estimator is unbiased ($\mathbb{E}[\hat{\pi}] = \pi$) and its uncertainty shrinks as $1/\sqrt{N}$. The three canonical quantities are defined in **equation (3)** below.

2. Requirements (intent, high level)

ID	Statement
R-01	The application shall estimate π by sampling points uniformly in the unit square $[0, 1]^2$ and counting the fraction inside the quarter circle $x^2 + y^2 \leq 1$.
R-02	The user shall specify the total number of samples N and the batch size B (samples computed per update tick).
R-03	The application shall show the running estimate $\hat{\pi}$ updating continuously as samples are processed.
R-04	The application shall show the current estimate’s absolute error against true π (<code>math.pi</code>) and its current standard error.
R-05	The application shall render the sampled points in a 2-D scatter with the quarter-circle boundary; hit and miss points are visually distinguishable.
R-06	The application shall render a convergence plot of $\hat{\pi}$ versus the number of samples, including a reference line at true π and confidence bands.
R-07	The user shall be able to Start, Pause, Resume, and Reset the estimation at any time.
R-08	The user shall be able to supply an optional RNG seed to make a run reproducible.
R-09	The GUI must remain responsive while millions of samples are being computed.
R-10	All heavy computation must occur off the Qt main (event-loop) thread.
R-11	The project shall be reproducible via <code>uv</code> on Python 3.12.

3. Behavior and state model

3.1 System state machine (AppState)



State	Meaning	Allowed controls enabled
IDLE	No estimation in progress; a fresh run may begin.	Start, Reset
RUNNING	Worker is actively producing samples.	Pause, Reset
PAUSED	Worker halted mid-run; progress retained.	Resume, Reset
COMPLETED	<code>processed == N</code> ; a terminal summary is shown.	Reset, Start (new run)
ERROR	A recoverable fault occurred; user can Recover/Reset.	Reset, Start

Transitions (event to target):

Event	From	To	Side-effect
Start(N,B,seed)	IDLE, COMPLETED, ERROR, PAUSED (via new run)	RUNNING	reset counters, spin worker, enable Pause/Reset
Pause	RUNNING	PAUSED	halt worker sampling, retain counters
Resume	PAUSED	RUNNING	continue worker
Reset	any	IDLE	stop worker, clear charts/counters, restore default input validity
RunComplete	RUNNING	COMPLETED	finalize summary
WorkerCrash	RUNNING	ERROR	capture exception text, finalize

Event	From	To	Side-effect
InputInvalid	any	(no state change)	disable Start, show validation message (see s8)

Constraints on transitions (see invariants s6): a new **Start** while **RUNNING/PAUSED** must first stop the current worker (equivalent to **Reset-then-Start**); it must never create a second live worker.

3.2 Per-tick sampling behavior (the hot loop)

```

For each tick while RUNNING and processed < N:
    k = min(B, N - processed)           # bounded last batch
    draw k (x, y) with x, y ~ U[0,1)    # seeded RNG, vectorized
    hits_k = count(x*x + y*y <= 1)      # the only hit rule
    total_hits += hits_k
    processed += k
    pi_hat = 4 * total_hits / processed
    emit Progress(processed, pi_hat, points[k]) # throttled (C-03)
if processed == N: emit RunComplete

```

3.3 Threading model

- **UI thread (main):** widget event loop only; all chart updates.
- **Worker thread:** owns the **MonteCarloEngine**; never touches widgets.
- Communication is Qt signals only (**QThread** plus **pyqtSignal**), plus a thread-safe stop flag guarded by a **QAtomicInt/lock**. No shared mutable buffers cross threads except via queued signals.

3.4 The estimator and its uncertainty

By the method of sections, the probability that a uniform point lands inside the quarter circle is $p = \pi/4$; the estimator, its standard error, and its variance are therefore (with h the running hit count and N the running sample count):

$$\hat{\pi} = 4 \cdot \frac{h}{N}, \quad \sigma_{\hat{\pi}} = 4 \sqrt{\frac{p(1-p)}{N}} \quad (p = h/N), \quad \text{Var}(\hat{\pi}) = \frac{16 p(1-p)}{N}$$

These are the formulas implemented by **estimate** / **standard_error** in C-01, enforced as invariants I-003 / I-004, and plotted in C-06.

4. Interfaces / contracts

C-01 MonteCarloEngine (pure, no GUI import — testable headless)

```

class MonteCarloEngine:
    __init__(self, seed: int | None = None) -> None
        # creates numpy Generator (np.random.default_rng(seed)).
        # seed=None -> nondeterministic; fixed seed -> reproducible (I-006).

    run_batch(self, n: int) -> BatchResult
        # Precondition: n >= 0.
        # Postcondition: returns this batch's draws and the running totals after
        #   incorporating the batch. Advances the engine's RNG forward.
        # Returns BatchResult:

```

```

# processed: int          # cumulative samples, monotonically nondecreasing
# total_hits: int        # cumulative hits
# hits_this_batch: int
# x: np.ndarray          # shape (n,), in [0,1)
# y: np.ndarray          # shape (n,), in [0,1)
# is_hit: np.ndarray     # bool; True iff x*x + y*y <= 1.0

@property
def estimate(self) -> float | None:
    # pi_hat = 4 * total_hits / processed, if processed > 0 else None

@property
def standard_error(self) -> float | None:
    # se = 4 * sqrt(p*(1-p)/processed), with p = total_hits / processed

```

C-02 EstimationWorker(QThread)

```

signals:
    progress(Progress)          # emitted per throttled tick
    completed()                 # emitted exactly once when processed == N
    crashed(str message)        # emitted on any uncaught worker exception

public slots / methods (main-thread, thread-safe to call):
    start_run(n_total: int, batch: int, interval_ms: int) -> None
    pause() -> None
    resume() -> None
    stop() -> None              # idempotent; safe to call from any state

```

The progress payload Progress is { processed:int, estimate:float, error:float, standard_error:float, batch_x:np.ndarray, batch_y:np.ndarray, batch_is_hit:np.ndarray }.

C-03 Throttle contract

The worker **computes** at full speed but **emits progress** no more often than every `interval_ms` (default 80 ms; ≈ 12 fps). Each emission carries at most the points accumulated since the last emission. This decouples *computation* from *visualization* and is the mechanism that guarantees R-09 / constraint K-01.

C-04 Data structures

```

Progress: processed:int, estimate:float, error_abs:float,
          standard_error:float, batch_x/f64, batch_y/f64, batch_is_hit/bool
Summary:  n_total:int, estimate:float, error_abs:float,
          standard_error:float, z_score:float    (z = error_abs / standard_error)

```

5. UI specification

5.1 Top-level layout (MainWindow)

```

+-----+
| Title: Monte Carlo pi Estimator |
+-----+-----+-----+
| LEFT COLUMN (controls+stats) | RIGHT COLUMN (plots) | | |
| N (spin)                     | +-----+ |
| Batch B (spin)                | | Scatter (C-05) | |

```

```

| Update interval ms (spin) | | quarter circle, || |
| Seed (int, optional) | | hits vs misses || |
| [Start] [Pause] [Resume] [Rst] | +-----+ |
| State label | +-----+ |
| pi_hat / true pi / error | | Convergence (C-06) || |
| samples processed / N | | pi_hat(n) || |
| standard error / z-score | | with +/- k sigma bands|| |
| validation message (inline) | +-----+ |
+-----+-----+-----+

```

C-05 Scatter plot (matplotlib, embedded via FigureCanvasQTAgi)

- Axes limits fixed to $x \in [0, 1]$, $y \in [0, 1]$; equal aspect.
- Quarter-circle arc $x^2 + y^2 = 1$ is drawn.
- Hit points and miss points use **two distinct colors** and are toggle-able.
- **Memory bounded (K-02)**: the scatter keeps at most `MAX_PLOTTED` (default 60_000) points — a subsample of the most recent draws — so memory and the `set_data` cost stay bounded for very large N .

C-06 Convergence plot

- x-axis = $\log_{10}(\text{processed})$; y-axis = $\hat{\pi}$.
- Series: running $\hat{\pi}$; a dashed reference line at `math.pi`; and shaded bands at $\pm 1\sigma$, $\pm 2\sigma$, $\pm 3\sigma$ computed from the **theoretical** standard error so that they are exact and not self-reinforcing:

$$\text{band}_k = \pi \pm k \sigma_{\hat{\pi}}(N) = \pi \pm k \cdot 4 \sqrt{\frac{(\pi/4)(1 - \pi/4)}{N}}$$

- x auto-scaled to the current `processed`.

5.2 Control widgets and validation

Widget	Type	Range / constraints	Invalid behavior (s8)
N	QSpinBox/QDoubleSpinBox	1 ... 1,000,000,000	disable Start, show message N must be >= 1
Batch B	QSpinBox	1 ... 10,000,000	show message Batch must be >= 1
Update interval	QSpinBox (ms)	0 ... 1000	clamp to [0, 1000]; 0 = “as fast as possible”
Seed	QLineEdit(int, optional)	integer, or blank	non-integer -> message, disable Start; blank -> seed = None
Start / Pause / Restore / Reset	QPushButton	enabled per state table (s3.1)	–

6. Invariants (must hold in every valid implementation)

ID	Invariant	Verified by
I-001	Every generated point satisfies $0 \leq x < 1$ and $0 \leq y < 1$ (the unit square).	T-05
I-002	A point is a hit iff $x^2 + y^2 \leq 1.0$. This is the <i>only</i> hit rule.	T-02
I-003	$\text{estimate} = 4h/\text{processed}$ whenever $\text{processed} > 0$; else None .	T-03
I-004	$\text{standard_error} = 4\sqrt{p(1-p)/\text{processed}}$, $p = h/\text{processed}$; else None .	T-03
I-005	processed and total_hits are monotonically nondecreasing; $\text{total_hits} \leq \text{processed}$.	T-04
I-006	Fixed seed implies an identical sequence, hence an identical estimate for the same N .	T-01
I-007	At most one live EstimationWorker exists at any time.	T-06
I-008	The UI thread never executes sampling code; all sampling is in the worker thread. (Design invariant; checked by the T-07 smoke test via thread identity.)	T-07
I-009	$\text{error_abs} \geq 0$ and $\text{z_score} = \text{error_abs}/\text{standard_error}$ (finite).	T-03

These invariants *constrain the agent’s solution space*: any implementation, however generated, that violates them is a defect even if it “works” on happy-path input.

7. Constraints (precise and measurable — not prose)

ID	Constraint	Measurement
K-01	While RUNNING , the UI must respond to a user action within $p_{95} < 50$ ms.	T-07 (event-loop liveness under a 10^7 -sample run, offscreen platform)
K-02	Maximum points retained for the scatter $\leq 60_000$ regardless of N ; peak plotting memory must not scale with N .	T-08
K-03	Default update interval is 80 ms; 0 means unthrottled (emit every batch).	T-04
K-04	Sampling throughput for $N \leq 10^7$ must not fall below $\sim 10^6$ samples/s on reference hardware. (Informative; soft)	T-07
K-05	No wall-clock blocking on the UI thread for any single progress handler.	T-07

8. Edge cases and failure semantics

ID	Situation	Required behavior
E-01	$N = 0$ or invalid	Start disabled; inline message N must be ≥ 1 ; no worker spun.
E-02	$B > N$	The last batch is clamped to $k = \min(B, N - \text{processed})$; the run still completes exactly at N . (I-003)
E-03	Interval = 0	Emit progress every batch (no throttle); the UI must still stay responsive via the worker thread.

ID	Situation	Required behavior
E-04	Resume/Resume or Pause/Pause (redundant)	No-op; no state corruption.
E-05	Start while RUNNING/PAUSED	Stop the current worker first; exactly one live worker (I-007); counters reset.
E-06	Reset while RUNNING	Immediately stop the worker, restore IDLE, clear plots/stats.
E-07	Worker exception (e.g., malformed input)	Emit <code>crashed(msg)</code> ; state $\rightarrow$ <code>ERROR</code> ; message shown; Start/Reset remain available. Never fabricate a result.
E-08	Very large N (10^9)	Bounded memory via K-02; the convergence x-axis is log-scaled; no overflow.
E-09	processed $< B$ at the tail	The partial final batch is computed; <code>completed</code> is emitted exactly once.
E-10	Window closed mid-run	<code>stop()</code> is called and joined; clean exit, no hang.
E-11	No display (CI)	Tests use <code>QT_QPA_PLATFORM=offscreen</code> ; app logic is unaffected.

Failure philosophy (from spec-engineering doctrine): failure behavior is *specified*, not left to the happy path. The dominant failure of a GUI is a *frozen event loop*; the dominant failure of a stochastic estimator is *presenting a false-precise number*. This specification forbids both: K-01 and K-05 forbid freezing, and the UI must always show `standard_error/z_score` so that an estimate is never read as exact.

- **Never:** freeze the UI; report a single π number without its uncertainty; keep the plot growing unbounded; create two live workers.

9. Acceptance criteria, tests, and evals

All tests target **Level-3 executable** criteria. GUI tests run offscreen via `pytest-qt` (`qt_qpa_platform = "offscreen"`, see `pyproject.toml`).

9.1 Engine tests (deterministic, no Qt) — fast, run always

ID	Criterion
T-01	Determinism (I-006): <code>estimate</code> for $N = 10^6$ with <code>seed=42</code> equals, bitwise, a second independent run with <code>seed=42</code> .
T-02	Hit rule (I-002): for a hand-built set of (x, y) , <code>is_hit</code> matches $x^2 + y^2 \leq 1$ exactly.
T-03	Formula invariants (I-003, I-004, I-009): after a batch, <code>estimate == 4*h/processed</code> , <code>standard_error == 4*sqrt((1-p)*p/processed)</code> , and <code>error_abs >= 0</code> .
T-04	Monotonicity and clamp (I-005, E-02): across batches <code>processed</code> is nondecreasing, <code>total_hits <= processed</code> , and a final batch clamps to the remainder.
T-05	Domain bound (I-001): all generated $x, y \in [0, 1)$.

9.2 Statistical / probabilistic evals (the “estimator is correct” checks)

ID	Criterion (property over random seeds)
T-09	Bias ≈ 0: $\mathbb{E}[\text{estimate}]$ over ≥ 50 seeds at $N = 2 \times 10^5$ lies within $2\bar{\sigma}$ of <code>math.pi</code> , where $\bar{\sigma} = \text{SE}/\sqrt{\text{trials}}$.
T-10	Variance matches theory: the empirical variance of <code>estimate</code> over ≥ 50 seeds at $N = 10^5$ lies within 20% of $16p(1-p)/N$, with $p = \pi/4$.
T-11	Convergence (variance decay): the standard error halves when N quadruples (ratio in $[0.45, 0.55]$, i.e. $\propto 1/\sqrt{N}$).
T-12	Empirical 3σ band: over ≥ 30 seeds at $N = 10^4$, at least 90% of runs satisfy $ \text{error} \leq 3\text{SE}$ (Monte-Carlo coverage spot-check).

9.3 GUI / integration tests (offscreen, pytest-qt)

ID	Criterion
T-06	Single worker (I-007 / E-05): rapid Start->Start yields exactly one live worker; the old one finished.
T-07	Liveness (K-01 / I-008): with a 10^7 -sample run in progress offscreen, the event loop still services a posted task within 50 ms; the sampling thread $\neq$ the main thread.
T-08	Bounded memory (K-02 / E-08): the scatter retains $\leq 60_000$ points after $N = 10^6$.
T-13	State machine (s3.1): Idle->Start->Running->(RunComplete)->Completed; Pause/Resume reachable; Reset returns to Idle; control-enable flags match the table.
T-14	Reproduction (R-08): a fixed seed in the UI reproduces the T-01 estimate after N samples.
T-15	Validation (E-01): $N < 1$ and a non-integer seed disable Start and show the message.

9.4 Manual / smoke eval (not automated; recorded)

- Visually confirm the scatter shows a clear circular boundary between the hit/miss regions and the convergence line settles within the $\pm 3\sigma$ bands (E-01, qualitative).
- **E-02** throughput sanity on reference hardware (informative).

10. Dependencies and environment

Concern	Decision	Rationale
Package/env manager	uv	Fast, reproducible, pins Python; satisfies R-11.
Python	3.12 ($3.12 \leq \text{ver} < 3.13$)	Requested; LTS-class; stable PyQt5 + numpy + matplotlib.
GUI	PyQt5 (5.15)	Requested.
Plotting	matplotlib (Qt5Agg backend, embedded <code>FigureCanvasQTAgg</code>)	Requested; integrates into Qt.
Numerics	numpy (vectorized sampling)	Fast batched RNG; required to make K-04 feasible.
Dev deps	pytest, pytest-qt, ruff	Automated Level-3 tests (s9) plus lint.
GUI test backend	QT_QPA_PLATFORM=offscreen	Headless CI without a display (E-11).

Concern	Decision	Rationale
---------	----------	-----------

Reproducibility commands (see README.md):

```
uv sync          # create .venv (Python 3.12), install everything
uv run pytest    # run the s9 test / eval suite
uv run monte-carlo-pi # launch the GUI
```

11. Traceability matrix (id -> where realized)

```
R-01 --> C-01(engine), C-05(scatter), I-002 --> T-02
R-03 --> C-02(signal)      --> C-06(plot), progress --> T-13
R-04 --> C-04(summary), I-003/004/009 --> T-03, T-12
R-06 --> C-06(convergence plot) --> T-08, E-01(visual)
R-07 --> state table (s3.1) --> T-13
R-08 --> C-01(seed) --> T-01, T-14
R-09/010 --> worker thread, C-03(throttle), K-01/05 --> T-07
R-11 --> deps, pyproject.toml, .python-version (s10) --> uv sync
I-005/E-02 --> C-01(run_batch clamp) --> T-04
I-007/E-05 --> C-02(start_run) --> T-06
E-01/E-15 --> validation --> T-15
```

Open questions / ambiguities flagged for the human (specification elicitation):

1. Which π ? — `math.pi` is assumed as the reference “true” value. (confirm)
2. Hit boundary at equality, $x^2 + y^2 = 1$ — included (measure-zero; does not affect the estimate). (confirm)
3. Convergence bands: theoretical versus empirical σ . The spec chooses **theoretical** (exact, non-self-reinforcing). (confirm)
4. The N cap of 10^9 is a UI convenience, not a mathematical limit; raise if needed.

End of specification. This document is the source of truth; implementation and tests are to be derived from it and kept in sync per s11.